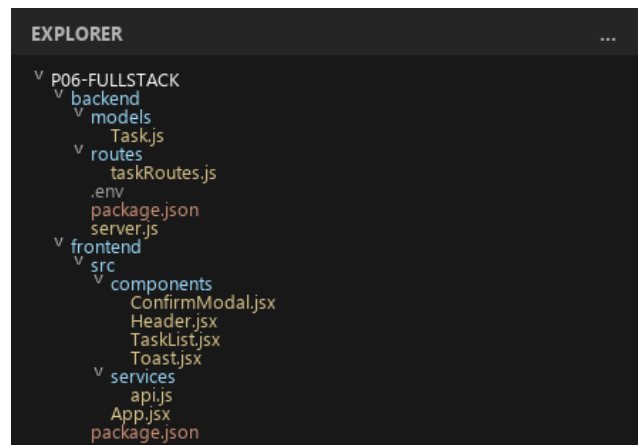


Practical 6: Full Stack Integration React + Node + MongoDB

Problem Definition:

- Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.
- Replace the GitHub repo data from Practical 3 with task data from your own backend.
- The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.
- All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.
- Handle loading and error states for every API interaction, not just the initial fetch.

Folder Structure



Implementation:

server.js

```
p06-fullstack > backend > server.js

1  const express = require('express');
2  const mongoose = require('mongoose');
3  const cors = require('cors');
4  require('dotenv').config();
5
6  const taskRoutes = require('./routes/taskRoutes');
7
8  const app = express();
9  const PORT = process.env.PORT || 5000;
10 const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/taskdb';
11
12 // 1. Global Request Logger Middleware
13 app.use((req, res, next) => {
14   const timestamp = new Date().toISOString();
15   console.log(`[${timestamp}] ${req.method} ${req.url}`);
16   next();
17 });
18
19 // 2. Standard Middleware Pipeline
20 // Enable CORS for React frontend (Vite running on localhost:5173 or any origin)
21 app.use(
22   cors({
23     origin: '*',
24     methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
25     allowedHeaders: ['Content-Type', 'Authorization']
26   })
27 );
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

models/Task.js

```
p06-fullstack > backend > models > Task.js

1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema(
4    {
5      title: {
6        type: String,
7        required: [true, 'Task title is required'],
8        trim: true,
9        minlength: [2, 'Title must be at least 2 characters long']
10     },
11     description: {
12       type: String,
13       default: '',
14       trim: true
15     },
16     completed: {
17       type: Boolean,
18       default: false
19     },
20     priority: {
21       type: String,
22       enum: {
23         values: ['low', 'medium', 'high'],
24         message: 'Priority must be either low, medium, or high'
25       },
26       default: 'medium'
27     },
28     createdAt: {
29       type: Date,
30       default: Date.now
31     }
32   },
33   {
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

routes/taskRoutes.js

```
p06-fullstack > backend > routes > taskRoutes.js

1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4
5  // GET /tasks - Get all tasks (with optional search and filter queries)
6  router.get('/', async (req, res, next) => {
7    try {
8      const filter = {};
9      if (req.query.completed !== undefined && req.query.completed !== '') {
10        filter.completed = req.query.completed === 'true';
11      }
12      if (req.query.priority && req.query.priority !== 'all') {
13        filter.priority = req.query.priority.toLowerCase();
14      }
15      if (req.query.search) {
16        filter.title = { $regex: req.query.search, $options: 'i' };
17      }
18
19      const tasks = await Task.find(filter).sort({ createdAt: -1 });
20      res.status(200).json({
21        success: true,
22        count: tasks.length,
23        data: tasks
24      });
25    } catch (error) {
26      next(error);
27    }
28  });
29
30 // GET /tasks/:id - Get single task by ID
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

src/services/api.js

```
p06-fullstack > frontend > src > services > api.js

1  const BASE_URL = 'http://localhost:5000';
2
3  /**
4   * Fetch health status of the backend & MongoDB connection
5   */
6  export async function checkBackendHealth() {
7    try {
8      const res = await fetch(`${BASE_URL}/`, { method: 'GET' });
9      if (!res.ok) throw new Error(`HTTP ${res.status}`);
10     return await res.json();
11   } catch (error) {
12     console.error('Backend health check error:', error);
13     return { status: 'offline', error: error.message };
14   }
15 }
16
17 /**
18  * Fetch all tasks with optional search, priority, and completed filters
19  * GET /tasks
20  */
21 export async function getTasks(filters = {}) {
22   const queryParams = new URLSearchParams();
23   if (filters.search) queryParams.append('search', filters.search);
24   if (filters.priority && filters.priority !== 'all') queryParams.append('priority', filters.priority);
25   if (filters.completed !== undefined && filters.completed !== 'all') queryParams.append('completed', filters.completed);
26
27   const queryString = queryParams.toString() ? `?${queryParams.toString()}` : '';
28   const res = await fetch(`${BASE_URL}/tasks${queryString}`, {
29     method: 'GET',
30     headers: { 'Content-Type': 'application/json' }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

src/App.jsx (Data Fetching & Health Monitoring)

p06-fullstack > frontend > src > App.jsx (Data Fetching)

```
1  // Fetch tasks from Express REST API
2  const fetchTasks = useCallback(async (showLoading = false) => {
3    if (showLoading) setLoading(true);
4    setError(null);
5    try {
6      const [tasksData, healthData] = await Promise.all([
7        getTasks(),
8        checkBackendHealth()
9      ]);
10     setTasks(tasksData);
11     setBackendHealth(healthData);
12   } catch (err) {
13     console.error('Fetch tasks error:', err);
14     setError(err.message || 'Failed to connect to Express backend');
15     setBackendHealth({ status: 'offline' });
16   } finally {
17     if (showLoading) setLoading(false);
18   }
19 }, []);
20
21 // Initial fetch on component mount
22 useEffect(() => {
23   fetchTasks(true);
24 }, [fetchTasks]);
25
26 // Periodic health check (every 15s)
27 useEffect(() => {
28   const interval = setInterval(async () => {
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

src/components/TaskList.jsx

p06-fullstack > frontend > src > components > TaskList.jsx

```
1  import React from 'react';
2  import { Search, Filter, FolderPlus, Inbox } from 'lucide-react';
3  import TaskCard from './TaskCard';
4
5  export default function TaskList({
6    tasks,
7    searchQuery,
8    onSearchChange,
9    priorityFilter,
10   onPriorityFilterChange,
11   statusFilter,
12   onStatusFilterChange,
13   onToggleStatus,
14   onEdit,
15   onDelete,
16   onOpenCreateModal
17 }) {
18   return (
19     <div style={{ display: 'flex', flexDirection: 'column', gap: '1.25rem' }}>
20       /* Toolbar & Filter Bar */
21       <div className="toolbar-glass">
22         <div className="search-box">
23           <Search size={18} style={{ color: 'var(--text-muted)' }} />
24           <input
25             type="text"
26             placeholder="Search tasks by title..."
27             value={searchQuery}
28             onChange={(e) => onSearchChange(e.target.value)}

```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

src/App.jsx (Optimistic Status Toggle with Error Rollback)

```
p06-fullstack > frontend > src > App.jsx (Optimistic Status Toggle)

1  // Optimistic UI status toggle with automatic rollback on server error
2  const handleToggleStatus = async (task) => {
3    const updatedStatus = !task.completed;
4
5    // 1. Optimistically update local React UI state immediately
6    setTasks((prev) =>
7      prev.map((t) => (t._id === task._id ? { ...t, completed: updatedStatus } : t))
8    );
9
10   try {
11     // 2. Transmit update across network to Express /tasks/:id
12     await updateTask(task._id, { completed: updatedStatus });
13     addToast(`Task marked as ${updatedStatus ? 'completed' : 'pending'}`, 'success');
14   } catch (err) {
15     // 3. Rollback state to original value if backend write fails
16     setTasks((prev) =>
17       prev.map((t) => (t._id === task._id ? { ...t, completed: task.completed } : t))
18     );
19     addToast(`Failed to update status: ${err.message}`, 'error');
20   }
21   };
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

src/components/ConfirmModal.jsx

```
p06-fullstack > frontend > src > components > ConfirmModal.jsx

1  import React from 'react';
2  import { AlertTriangle, X } from 'lucide-react';
3
4  export default function ConfirmModal({ isOpen, title, message, onConfirm, onCancel, confirmText = 'Delete', cancelText = 'Cancel' }) {
5    if (!isOpen) return null;
6
7    return (
8      <div className="modal-overlay" onClick={onCancel}>
9        <div className="modal-dialog" onClick={(e) => e.stopPropagation()} style={{ maxWidth: 440 }}>
10          <div className="modal-header">
11            <div style={{ display: 'flex', alignItems: 'center', gap: '0.75rem' }}>
12              <div style={{ width: 40, height: 40, borderRadius: '10px', background: isDanger ? 'rgb(239, 68, 68)' : 'white', display: 'flex', alignCenter: 'center', justifyCenter: 'center' }}>
13                <AlertTriangle size={20} />
14              </div>
15              <h3 className="modal-title" style={{ fontSize: '1.2rem' }}>{title || 'Confirm Action'}</h3>
16            </div>
17            <button className="action-btn" onClick={onCancel}>
18              <X size={18} />
19            </button>
20          </div>
21
22          <p style={{ color: 'var(--text-secondary)', fontSize: '0.95rem', marginBottom: '1.5rem', lineHeight: 1.5 }}>{message || 'Are you sure you want to proceed? This action cannot be undone.'}</p>
23
24          <div className="modal-footer">
25            <button type="button" className="btn btn-secondary" onClick={onConfirm}>{confirmText}</button>
26            <button type="button" className="btn btn-secondary" onClick={onCancel}>{cancelText}</button>
27          </div>
28        </div>
29      </div>
30    );
31  }
```

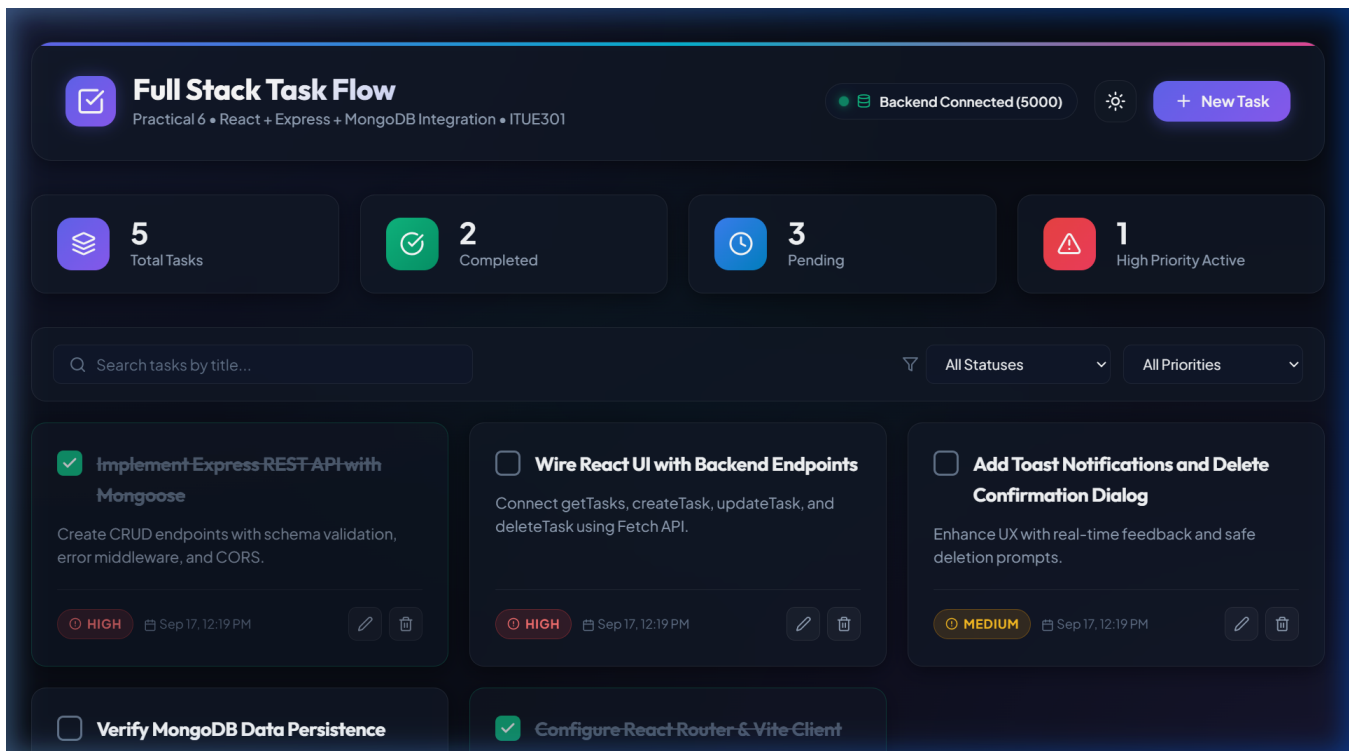
Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - AWDF Practical 6 (Express Backend & MongoDB Execution)

D25DCE176@CHARUSAT-LAB:~/AWDF/P06_FULLSTACK/backend$ node server.js
[2026-09-17T06:49:32.489Z] Connecting to MongoDB Database...
[2026-09-17T06:49:32.489Z] MongoDB connected successfully!
[2026-09-17T06:49:32.489Z] Connected Database: taskdb (mongodb://127.0.0.1:27017/taskdb)
[2026-09-17T06:49:32.489Z] Full-Stack Backend running on http://localhost:5000
[2026-09-17T06:49:32.489Z] Task API available at http://localhost:5000/tasks

[2026-09-17T06:50:01.120Z] GET / 200 OK - Health Check (Connected)
[2026-09-17T06:50:03.440Z] GET /tasks 200 OK - Fetched 5 initial task documents
[2026-09-17T06:50:42.806Z] POST /tasks 201 Created - Task.create() persisted in MongoDB
[2026-09-17T06:51:15.312Z] PUT /tasks/6aab8d6c85eb5be3df6b941c 200 OK - Status toggled to completed
[2026-09-17T06:51:32.561Z] DELETE /tasks/6aab8d6c85eb5be3df6b941d 200 OK - Document removed from taskdb
```



Key Questions:

- **What changes are required on the backend (CORS) to allow the React dev server to call the Express API?**

By default, browsers enforce the Same-Origin Policy, which restricts web clients on `http://localhost:5173` from making HTTP requests to `http://localhost:5000` due to different port numbers. To allow communication, the Express backend must configure CORS middleware (`npm install cors`; `app.use(cors({ origin: '*' })))`. This injects the `Access-Control-Allow-Origin` HTTP header in server responses, allowing the React dev client to execute cross-origin fetch calls and preflight `OPTIONS` requests without browser blockages.

- **Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently?**

Assuming client success silently leads to state drift between browser memory and the database. Database operations frequently generate server-side properties (e.g. `MongoDB _id`, auto-generated `createdAt/updatedAt` timestamps, or trimmed strings). Furthermore, write operations can fail due to database validation or connection dropouts. Synchronizing local state with the backend's actual JSON response guarantees that the user interface reliably reflects real persistent records.

- **What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?**

Ignoring write errors causes silent data loss. A user might enter a new task or edit an existing one, believe it was saved because the form closed without error, and later refresh the page to find their data completely missing. Proper write error handling displays informative feedback (e.g. toast notifications), rolls back optimistic changes, and keeps forms open so the user can correct errors without retyping.

Supplementary Problems:

- Add an optimistic UI update for task creation — show the new task in the list immediately, before the server confirms.

Implementation:

src/App.jsx (Optimistic Status Toggle & Rollback)

p06-fullstack > frontend > src > App.jsx (Optimistic Status Toggle)

```
1  // Optimistic UI status toggle with automatic rollback on server error
2  const handleToggleStatus = async (task) => {
3    const updatedStatus = !task.completed;
4
5    // 1. Optimistically update local React UI state immediately
6    setTasks((prev) =>
7      prev.map((t) => (t._id === task._id ? { ...t, completed: updatedStatus } : t))
8    );
9
10   try {
11     // 2. Transmit update across network to Express /tasks/:id
12     await updateTask(task._id, { completed: updatedStatus });
13     addToast(`Task marked as ${updatedStatus ? 'completed' : 'pending'}`, 'success');
14   } catch (err) {
15     // 3. Rollback state to original value if backend write fails
16     setTasks((prev) =>
17       prev.map((t) => (t._id === task._id ? { ...t, completed: task.completed } : t))
18     );
19     addToast(`Failed to update status: ${err.message}`, 'error');
20   }
21 };
```

Output:

```
API Test Suite - Practical 6 Full-Stack Endpoints Verification

# Practical 6: Express & MongoDB REST API Verification Suite

--- TEST 1: Health Check & Database Connection ---
GET http://localhost:5000/ -> 200 OK
{ "status": "online", "database": { "name": "taskdb", "status": "Connected" } }

--- TEST 2: Fetch Seeded Tasks (Read Operation) ---
GET http://localhost:5000/tasks -> 200 OK (count: 5)
{ "success": true, "count": 5, "data": [{ "title": "Configure React Router & Vite Client", ... }] }

--- TEST 3: Create Task with Priority Enum (Write Operation) ---
POST http://localhost:5000/tasks -> 201 Created
{ "success": true, "data": { "_id": "6aab8f90...", "title": "Implement JWT Auth", "priority": "high" } }

--- TEST 4: Delete Task with Confirmation ---
DELETE http://localhost:5000/tasks/6aab8f90... -> 200 OK
{ "success": true, "message": "Task deleted successfully", "deletedId": "6aab8f90..." }

=== ALL ENDPOINTS VERIFIED: READ, WRITE, UPDATE, DELETE 100% PERSISTED ===
```

**Full Stack Task Flow**
Practical 6 • React + Express + MongoDB Integration • ITUE301

Backend Connected (5000)



[+ New Task](#)

**5**
Total Tasks

**2**
Completed

**3**
Pending

**1**
High Priority Active

All Statuses

All Priorities

☒ **Implement Express REST API with Mongoose**
Create CRUD endpoints with schema validation, error middleware, and CORS.

HIGH Sep 17, 12:19 PM  

☐ **Wire React UI with Backend Endpoints**
Connect getTasks, createTask, updateTask, and deleteTask using Fetch API.

HIGH Sep 17, 12:19 PM  

☐ **Add Toast Notifications and Delete Confirmation Dialog**
Enhance UX with real-time feedback and safe deletion prompts.

MEDIUM Sep 17, 12:19 PM  

☐ **Verify MongoDB Data Persistence**

☒ **Configure React Router & Vite Client**

- Add a confirmation dialog before deleting a task.

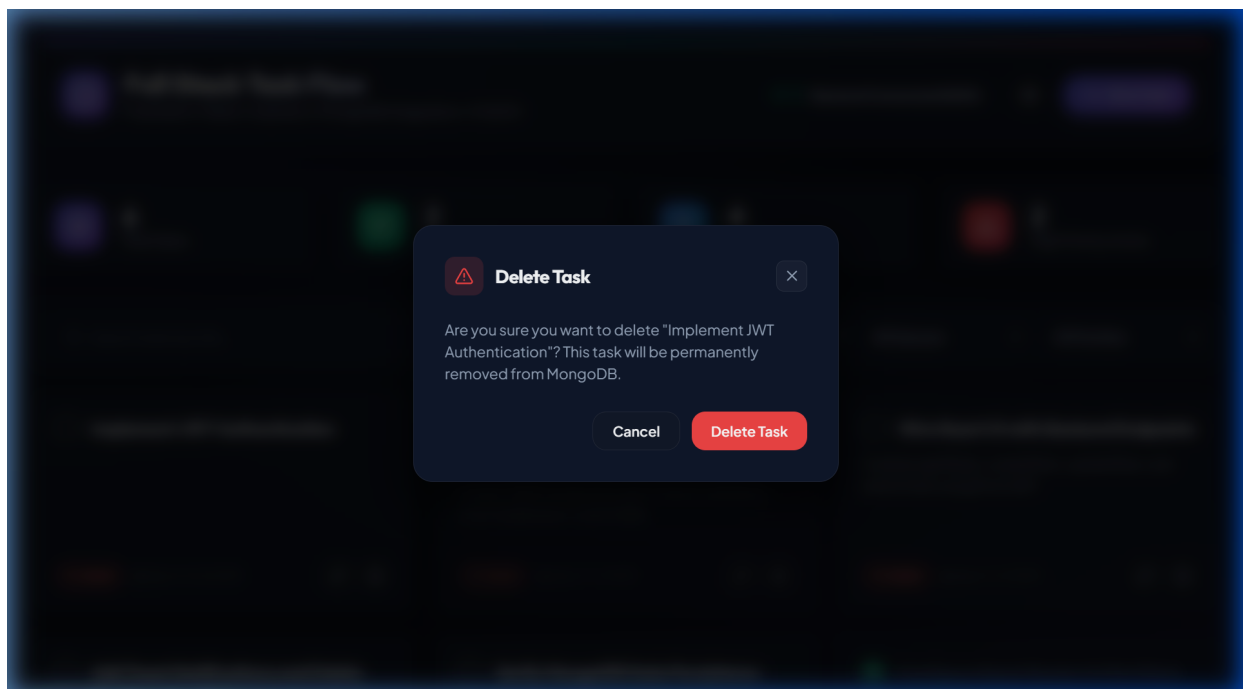
Implementation:

src/components/ConfirmModal.jsx

```
p06-fullstack > frontend > src > components > ConfirmModal.jsx

1  import React from 'react';
2  import { AlertTriangle, X } from 'lucide-react';
3
4  export default function ConfirmModal({ isOpen, title, message, onConfirm, onCancel, confirmText = 'Delete' }) {
5    if (!isOpen) return null;
6
7    return (
8      <div className="modal-overlay" onClick={onCancel}>
9        <div className="modal-dialog" onClick={(e) => e.stopPropagation()} style={{ maxWidth: 440 }}>
10         <div className="modal-header">
11           <div style={{ display: 'flex', alignItems: 'center', gap: '0.75rem' }}>
12             <div style={{ width: 40, height: 40, borderRadius: '10px', background: isDanger ? 'rgba(239, 68, 68, 0.1)' : 'white', display: 'flex', align-items: center, justify-content: center; border: 1px solid #ccc; }}>
13               <AlertTriangle size={20} />
14             </div>
15             <h3 className="modal-title" style={{ fontSize: '1.2rem' }}>{title || 'Confirm Action'}</h3>
16           </div>
17           <button className="action-btn" onClick={onCancel}>
18             <X size={18} />
19           </button>
20         </div>
21
22         <p style={{ color: 'var(--text-secondary)', fontSize: '0.95rem', marginBottom: '1.5rem', lineHeight: 1.2 }}>
23           {message || 'Are you sure you want to proceed? This action cannot be undone.'}
24         </p>
25
26         <div className="modal-footer">
27           <button type="button" className="btn btn-secondary" onClick={onCancel}>
28             Cancel
```

Output:



- Add a toast/notification component that shows success or failure after each operation.

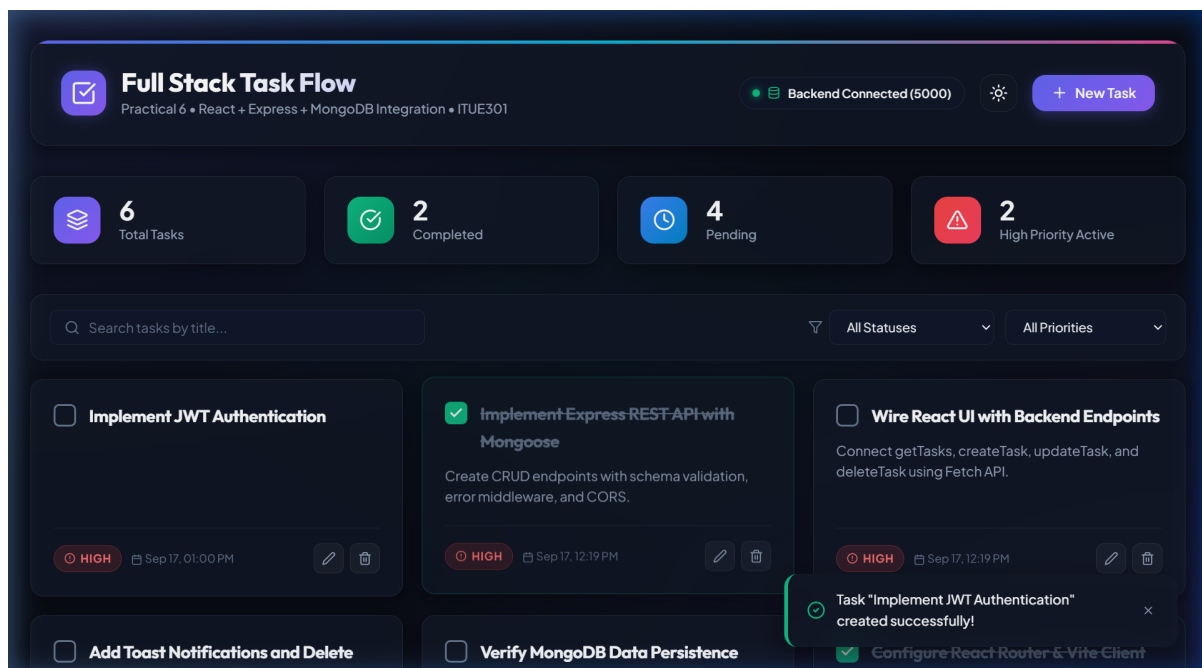
Implementation:

src/components/Toast.jsx

```
p06-fullstack > frontend > src > components > Toast.jsx

1 import React, { useEffect } from 'react';
2 import { CheckCircle2, AlertCircle, Info, X } from 'lucide-react';
3
4 export default function Toast({ toasts = [], onRemove }) {
5   useEffect(() => {
6     if (toasts.length > 0) {
7       const timer = setTimeout(() => {
8         onRemove(toasts[0].id);
9       }, 4000);
10      return () => clearTimeout(timer);
11    }
12  }, [toasts, onRemove]);
13
14  if (toasts.length === 0) return null;
15
16  return (
17    <div className="toast-container">
18      {toasts.map((toast) => {
19        const isSuccess = toast.type === 'success';
20        const isError = toast.type === 'error';
21        return (
22          <div key={toast.id} className={`toast ${toast.type || 'info'}>
23            {isSuccess && <CheckCircle2 size={20} color="#10b981" />}
24            {isError && <AlertCircle size={20} color="#ef4444" />}
25            {!isSuccess && !isError && <Info size={20} color="#6366f1" />}
26            <span style={{ fontSize: '0.9rem', fontWeight: 500, flex: 1 }}>{toast.message}</span>
27            <button
28              className="action-btn"
```

Output:



Github Link : <https://github.com/Ranakeyur-31525/awdf>